DATE 17/10/26

CUDA

S M T W T F S

Diff. b/w Shared Memory use & Memory Hierarchy use:

→ Shared Mem can hold a specific set of values which could be reused by Neighbouring threads to use that data for calculation.

→ It is local to a block & all threads inside the block can share that.

→ It will be implemented only if we ask to load a useful group of data into the shared Memory.

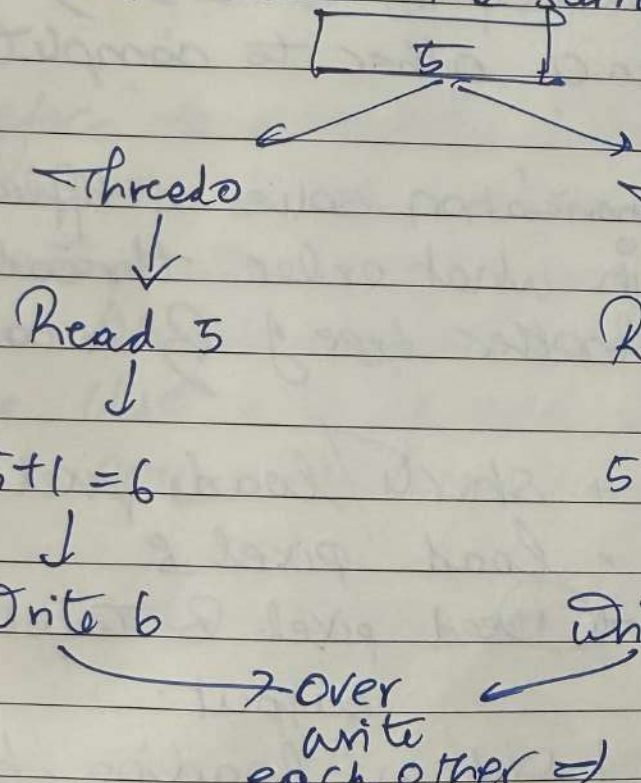
Whereas, Memory Hierarchy is what happens on default. If threads don't share data need, L1 cache → L2 cache → Global Memory.

DATE / /

S M T W T F S

Race Conditions & Synchronization:

Say there are two threads, which increments the current value by 1, they both access the same shared value



Actually, we wanted both threads to increment value by 1, so, actual answer should have been ⇒ 5+1+1=7, but, since, both Accessed & performed calc @ same time, we got only 6

⇒ Race Condition

DATE / / S M T W T F S

Therefore, to solve this problem, we use Syncronization.

We use __syncthreads(), which makes all threads participating in a block wait for each other to complete it processing.

But, synchonoriciation solves a different problem of in what order threads proceed. ∴ Another type of Race condition

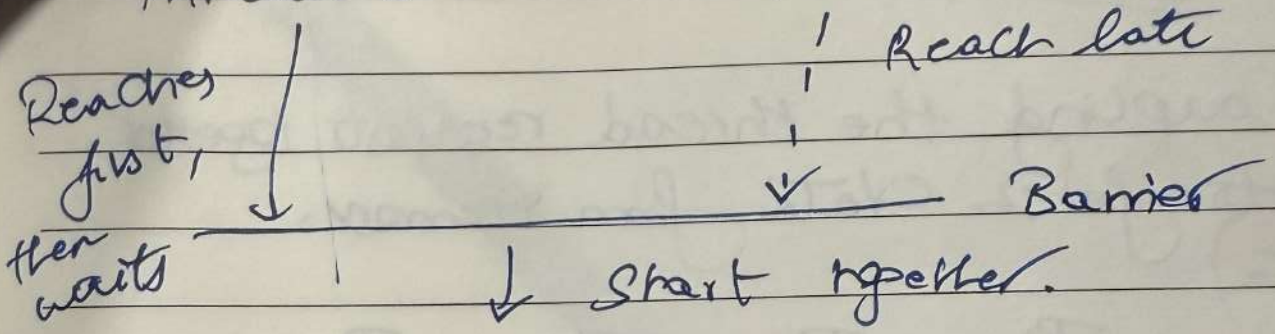
Ex) Thread 0 → starts loads pixel A
Thread 1 → load pixel B
Thread 0 → Need pixel B to calc. output.

But, Thread 1 delays loading, hence Thread 0 might use an old/unitialized value. Therefore, to solve this,

Syncronization creates a Barrier which has to be reached by both threads to start next step.

DATE / /

S M T W T F S
O O O O O O O



Therefore to actually avoid the Race condition which we saw, overwriting data, we could use: atomicAdd().

* Atomic operations, (ex) ~~Atomicsum~~.
* Use one thread for processing one data, (ex) 1 thread → 1 pixel

Hence, Race conditions

- Early reach → Syncronization
- overwriting → atomicAdd().

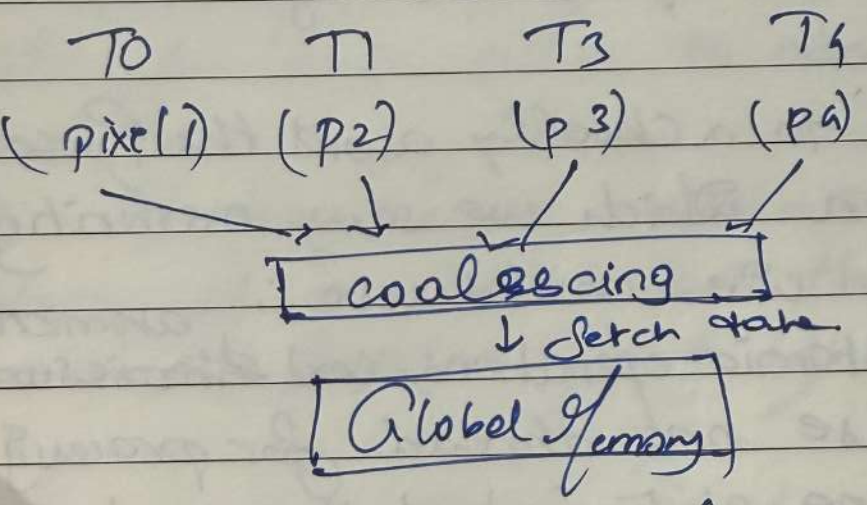
DATE / /

S M T W T F S
O O O O O O O

Coalescing:

Coupling the thread requests together to fetch data from Memory.

Ex)



It happens in Warp (32 threads)

Ex) Group of 64 threads → grouped into 2 warps

